EXP 1

```
[1]: W = [(0,1,2), (3,4,5), (6,7,8), (0,3,6), (1,4,7), (2,5,8), (0,4,8),
     (2,4,6)]
     def win(b, p): return any(all(b[i]==p for i in w) for w in W)

     def dfs(b, ai):
         if win(b, 'O'): return 1
         if win(b, 'X'): return -1
         if ' ' not in b: return 0
         s = [(dfs(b[:i] + ['O' if ai else 'X'] + b[i+1:], not ai), i) for i in
     range(9) if b[i] == ' ']
         return max(s)[0] if ai else min(s)[0]

     b = [' '] * 9
     while ' ' in b and not win(b, 'O') and not win(b, 'X'): 4
         b[int(input("Enter position (1-9): ")) - 1] = 'X'
         if ' ' in b and not win(b, 'X'):
             best_move = max([(dfs(b[:i] + ['O'] + b[i+1:], False), i) for i in
     range(9) if b[i] == ' '])[1]
             b[best_move] = 'O'
         print('\n-+-+-\n'.join(f"{b[i]}|{b[i+1]}|{b[i+2]}" for i in (0, 3, 6)))

     print("AI Wins" if win(b, 'O') else "Player Wins" if win(b, 'X') else
     "Draw")
```

```
Enter position (1-9):  8
 | |
-+-+-
 | |
-+-+-
 |X|O
Enter position (1-9):  3
 | |X
-+-+-
 |O|
-+-+-
 |X|O
Enter position (1-9):  4
O| |X
-+-+-
X|O|
-+-+-
 |X|O
AI Wins
```

EXP 2

```
[3]: def get_inversion_parity(state):
         tiles = [tile for tile in state if tile != 0]
         inversions = sum(
             1 for i in range(len(tiles))
             for j in range(i + 1, len(tiles))
             if tiles[i] > tiles[j]
         )
         return inversions % 2

     def is_reachable(state1, state2):
         # If parities match, they are in the same disjoint set
         return get_inversion_parity(state1) == get_inversion_parity(state2)

     state_A = [int(x) for x in input(
         "Enter State A separated by spaces (e.g., 1 2 3 4 5 6 7 8 0): "
     ).split()]

     state_B = [int(x) for x in input(
         "Enter State B separated by spaces: "
     ).split()]

     if is_reachable(state_A, state_B):
         print("\nResult: TRUE - They are in the SAME disjoint set (reachable from one another).")
     else:
         print("\nResult: FALSE - They are in DIFFERENT disjoint sets (impossible to reach).")
```

```
Enter State A separated by spaces (e.g., 1 2 3 4 5 6 7 8 0):  4
Enter State B separated by spaces:  3

Result: TRUE - They are in the SAME disjoint set (reachable from one another).
```

EXP3

```
# 1. Define facts & Store in Knowledge Base
knowledge_base = {
    "Alice": {"attendance": 80, "marks": 75},
    "Bob": {"attendance": 65, "marks": 80},
    "Charlie": {"attendance": 90, "marks": 45},
    "David": {"attendance": 78, "marks": 60}
}

# 2. Define predicates
def good_attendance(student):
    return knowledge_base[student]["attendance"] >= 75

def good_marks(student):
    return knowledge_base[student]["marks"] >= 50

# 3. Create rules & Match rules with facts
def is_eligible(student):
    return good_attendance(student) and good_marks(student)

# 4. Apply inference mechanism & Generate conclusions
print("--- Predicate Logic Evaluation ---")

for student in knowledge_base:
    status = "Eligible" if is_eligible(student) else "Not Eligible"
    print(f"{student}: {status}")
```

```
--- Predicate Logic Evaluation ---
Alice: Eligible
Bob: Not Eligible
Charlie: Not Eligible
David: Eligible
```

EXP 4

```
# 1. Load dataset (Attendance, Internal Marks, Eligibility)
dataset = [['High', 'Good', 'Yes'],
           ['High', 'Poor', 'No'],
           ['Low', 'Good', 'No'],
           ['High', 'Good', 'Yes']]

# 2. Separate attributes and target labels
X, Y = [d[:-1] for d in dataset], [d[-1] for d in dataset]

# 3. Initialize most specific hypothesis (S) and general (G)
S = X[0][:]
G = [['?'] * len(S)]

# 4. Read training examples
for i in range(len(X)):

    # 5. For positive examples: generalize hypothesis S, remove inconsistent G
    if Y[i] == 'Yes':
        S = [S[j] if S[j] == X[i][j] else '?' for j in range(len(S))]
        G = [g for g in G if all(g[k] == '?' or g[k] == X[i][k]
                                 for k in range(len(S)))]

    # 6. For negative examples: update/specialize boundaries of G
    else:
        new_G = []
        for g in G:
            for j in range(len(S)):
                if g[j] == '?' and S[j] != '?' and S[j] != X[i][j]:
                    new_G.append(g[:j] + [S[j]] + g[j+1:])
        G = new_G

print(f"Find-S Hypothesis: {S}\n")
print("Candidate Elimination Hypothesis")
print("Final Specific Hypothesis (S):", S)
print("Final General Hypothesis (G):", G)
```

```
Find-S Hypothesis: ['High', 'Good']

Candidate Elimination Hypothesis
Final Specific Hypothesis (S): ['High', 'Good']
Final General Hypothesis (G): [['High', 'Good']]
```

EXP 5

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

data = {
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Rain',
                'Overcast', 'Sunny', 'Sunny', 'Rain'],
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool', 'Cool',
                    'Mild', 'Mild', 'Mild'],
    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal',
                 'Normal', 'High', 'Normal', 'Normal'],
    'Windy': ['False', 'True', 'False', 'False', 'False', 'True', 'True',
              'False', 'False', 'True'],
    'PlayTennis': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'No',
                   'Yes', 'Yes']
}

df = pd.DataFrame(data)

X = pd.get_dummies(df.drop('PlayTennis', axis=1))  # Convert categorical to numeric
y = df['PlayTennis']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

clf = DecisionTreeClassifier(criterion='entropy', random_state=42)
clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}\n")

tree_rules = export_text(clf, feature_names=list(X.columns))
print(tree_rules)
```

```
Accuracy: 0.33

|--- Outlook_Sunny <= 0.50
|   |--- class: Yes
|--- Outlook_Sunny >  0.50
|   |--- class: No
```

EXP6

```
# ID3 Decision Tree: compare depth & overfitting on datasets
from sklearn.datasets import load_iris, load_wine
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

datasets = [("Iris", load_iris()), ("Wine", load_wine())]

for name, data in datasets:
    X_train, X_test, y_train, y_test = train_test_split(
        data.data, data.target, test_size=0.3, random_state=42)

    model = DecisionTreeClassifier(criterion="entropy")   # ID3
    model.fit(X_train, y_train)

    train_acc = model.score(X_train, y_train)
    test_acc = model.score(X_test, y_test)

    print(f"\nDataset: {name}")
    print("Tree Depth:", model.get_depth())
    print("Train Accuracy:", round(train_acc, 2))
    print("Test Accuracy :", round(test_acc, 2))

    if train_acc > test_acc:
        print("Possible Overfitting")
    else:
        print("Good Generalization / Underfitting")
```

```
Dataset: Iris
Tree Depth: 7
Train Accuracy: 1.0
Test Accuracy : 0.98
Possible Overfitting

Dataset: Wine
Tree Depth: 5
Train Accuracy: 1.0
Test Accuracy : 0.83
Possible Overfitting
```

7TH ONE

[14]:
```
X = [[1], [2], [3], [8], [9], [10]]
y = [0, 0, 0, 1, 1, 1]
# 1. SUPERVISED LEARNING

print("\nSUPERVISED LEARNING")

# Simple prediction using labeled data
def supervised_predict(value):

    if value < 5:
        return 0
    else:
        return 1

print("Prediction for 2 :", supervised_predict(2))
print("Prediction for 9 :", supervised_predict(9))

# 2. UNSUPERVISED LEARNING

print("\nUNSUPERVISED LEARNING")

# Simple clustering without labels
for value in X:

    if value[0] < 5:
        cluster = "Cluster A"
    else:
        cluster = "Cluster B"

    print(value[0], "->", cluster)

# 3. SEMI-SUPERVISED LEARNING

print("\nSEMI-SUPERVISED LEARNING")

# Some data labeled, some unlabeled
labeled_data = {1: 0, 9: 1}

unlabeled = [2, 3, 8, 10]

for value in unlabeled:

    if value < 5:
        predicted_label = 0
    else:
        predicted_label = 1

    print("Value :", value,
          "Predicted Label :", predicted_label)

# 4. REINFORCEMENT LEARNING

print("\nREINFORCEMENT LEARNING")

score = 0

actions = ["Correct", "Wrong", "Correct"]

for action in actions:

    if action == "Correct":
        score += 10
        print("Reward +10")
    else:
        score -= 5
        print("Penalty -5")

print("Final Score :", score)
```

```
SUPERVISED LEARNING
Prediction for 2 : 0
Prediction for 9 : 1

UNSUPERVISED LEARNING
1 -> Cluster A
2 -> Cluster A
3 -> Cluster A
8 -> Cluster B
9 -> Cluster B
10 -> Cluster B

SEMI-SUPERVISED LEARNING
Value : 2 Predicted Label : 0
Value : 3 Predicted Label : 0
Value : 8 Predicted Label : 1
Value : 10 Predicted Label : 1

REINFORCEMENT LEARNING
Reward +10
Penalty -5
Reward +10
Final Score : 15
```

8TH ONE

```
# Training data
data = [
    (['Sunny','Warm','Normal'], 'Yes'),
    (['Sunny','Warm','High'], 'Yes'),
    (['Rainy','Cold','High'], 'No'),
    (['Sunny','Warm','Normal'], 'Yes')
]

# Find-S
S = ['0'] * len(data[0][0])

for x, y in data:
    if y == 'Yes':
        for i in range(len(S)):
            if S[i] == '0':
                S[i] = x[i]
            elif S[i] != x[i]:
                S[i] = '?'

print("Find-S Hypothesis:", S)

# Candidate Elimination
G = [['?'] * len(S)]

for x, y in data:
    if y == 'Yes':
        G = [g for g in G if all(g[i] == '?' or g[i] == x[i]
                                 for i in range(len(x)))]
    else:
        newG = []
        for g in G:
            for i in range(len(x)):
                if g[i] == '?' and S[i] != x[i]:
                    h = g.copy()
                    h[i] = S[i]
                    newG.append(h)
        G = newG

print("Specific Boundary S:", S)
print("General Boundary G:", G)
```

```
Find-S Hypothesis: ['Sunny', 'Warm', '?']
Specific Boundary S: ['Sunny', 'Warm', '?']
General Boundary G: [['Sunny', '?', '?'], ['?', 'Warm', '?'], ['?', '?', '?']]
```

9TH ONE

```
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import RidgeCV
from sklearn.metrics import mean_squared_error, r2_score

# 1-4. Load Dataset & Handle Missing (SKlearn dataset is pre-cleaned)
housing = fetch_california_housing()
X, y = housing.data, housing.target

# 5-6. Feature Engineering (Scaling) & Train/Test Split
X_scaled = StandardScaler().fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

# 7-9. Train Model, Fine-Tune Parameters (done automatically by RidgeCV), and Predict
model = RidgeCV(alphas=[0.1, 1.0, 10.0]).fit(X_train, y_train)
y_pred = model.predict(X_test)

# 10. Evaluate Performance
mse = mean_squared_error(y_test, y_pred)
print(
    f"MSE: {mse:.4f} | RMSE: {np.sqrt(mse):.4f} | "
    f"R2 Score: {r2_score(y_test, y_pred):.4f}\n"
)

# Display Feature Weights
print("Feature Weights:")
for name, weight in zip(housing.feature_names, model.coef_):
    print(f"{name}: {weight:.4f}")
```

```
MSE: 0.5559 | RMSE: 0.7456 | R2 Score: 0.5758

Feature Weights:
MedInc: 0.8523
HouseAge: 0.1225
AveRooms: -0.3049
AveBedrms: 0.3708
Population: -0.0023
AveOccup: -0.0366
Latitude: -0.8959
Longitude: -0.8682
```

10TH

[20]:
```
import numpy as np

# Each row represents a simple image pattern
# Values:
# 0 = White pixel
# 1 = Black pixel
# 4x4 simplified digit patterns

X = np.array([

    # Digit 0
    [1,1,1,1,
     1,0,0,1,
     1,0,0,1,
     1,1,1,1],

    # Digit 1
    [0,1,0,0,
     1,1,0,0,
     0,1,0,0,
     1,1,1,0],

    # Digit 2
    [1,1,1,0,
     0,0,1,0,
     1,1,1,0,
     1,0,0,0],

    # Digit 3
    [1,1,1,0,
     0,0,1,0,
     1,1,1,0,
     0,0,1,0],

    # Digit 4
    [1,0,1,0,
     1,0,1,0,
     1,1,1,0,
     0,0,1,0]

])

# Labels for digits
y = np.array([0, 1, 2, 3, 4])

# ----------------------------------------------------
# DISPLAY DATASET
# ----------------------------------------------------

print("========== SIMPLE MNIST-LIKE DATASET ==========\n")

for i in range(len(X)):
    print("Digit Label :", y[i])

    # Convert 1D vector into 4x4 image
    image = X[i].reshape(4, 4)
    print(image)
    print()

# ----------------------------------------------------
# DATASET INFORMATION
# ----------------------------------------------------

print("Dataset Shape :", X.shape)
print("Number of Labels :", len(y))
```

```
========== SIMPLE MNIST-LIKE DATASET ========

Digit Label : 0
[[1 1 1 1]
 [1 0 0 1]
 [1 0 0 1]
 [1 1 1 1]]

Digit Label : 1
[[0 1 0 0]
 [1 1 0 0]
 [0 1 0 0]
 [1 1 1 0]]

Digit Label : 2
[[1 1 1 0]
 [0 0 1 0]
 [1 1 1 0]
 [1 0 0 0]]

Digit Label : 3
[[1 1 1 0]
 [0 0 1 0]
 [1 1 1 0]
 [0 0 1 0]]

Digit Label : 4
[[1 0 1 0]
 [1 0 1 0]
 [1 1 1 0]
 [0 0 1 0]]

Dataset Shape : (5, 16)
Number of Labels : 5
```